

RELATÓRIO TÉCNICO – PARTE PRÁTICA

Implementação Prática do MVP da Plataforma Pedidos Veloz

Curso: Engenharia Cloud DevOps

Aluno: João Pereira

Repositório: <https://github.com/jv0098/pedido-veloz>

Vídeo Pitch: <https://youtu.be/IO1Sqyp9Et4>

1. INTRODUÇÃO

Este relatório apresenta a implementação prática do MVP da plataforma Pedidos Veloz, abordando a containerização dos serviços, orquestração com Kubernetes, automação CI/CD, observabilidade e infraestrutura como código.

2. AMBIENTE LOCAL COM DOCKER

Foi utilizado o Docker Compose para padronizar o ambiente de desenvolvimento e facilitar a execução da aplicação.

Componentes

- **API Gateway (Nginx):** centraliza o acesso aos serviços.
- **Pedidos Service:** gerenciamento de pedidos.
- **Pagamento Service:** simulação de pagamentos.
- **Estoque Service:** controle de estoque.
- **PostgreSQL:** banco de dados persistente.

Boas Práticas

- Volume persistente para banco de dados.
- Rede isolada entre containers.
- Uso de secrets para credenciais.
- Healthchecks para garantir inicialização correta dos serviços.

Comando principal:

```
docker compose up -d --build
```

3. CONTEINERIZAÇÃO

Cada microsserviço utiliza Dockerfile Multi-stage para reduzir tamanho da imagem e melhorar a segurança.

Práticas adotadas

- Imagens base Alpine Linux.
- Execução sem privilégios administrativos (`USER node`).
- Versionamento das imagens por SHA do Git.
- Separação entre ambiente de build e runtime.

Essas medidas reduzem o consumo de recursos e aumentam a segurança operacional.

4. KUBERNETES

Os manifestos Kubernetes foram organizados na pasta `k8s/`.

Recursos implementados

- Deployment com múltiplas réplicas.
- Service para comunicação interna.
- ConfigMap para configurações.
- Secret para credenciais.
- Horizontal Pod Autoscaler (HPA).

Disponibilidade

Foram configuradas:

- **Liveness Probe:** monitora a saúde do container.
- **Readiness Probe:** controla quando o pod pode receber tráfego.

A estratégia de deploy adotada foi **Rolling Update**, permitindo atualizações sem indisponibilidade do sistema.

5. PIPELINE CI/CD

A automação foi implementada com GitHub Actions.

Etapas principais

1. Lint e Testes

- Validação de código.
- Execução de testes automatizados.

2. Build da Imagem

- Criação da imagem Docker.
- Publicação em registry.

3. Deploy

- Atualização dos manifestos.
- Aplicação automática no cluster Kubernetes.
- Possibilidade de rollback em caso de falha.

Essa abordagem reduz erros manuais e acelera a entrega contínua.

6. OBSERVABILIDADE E ESCALABILIDADE

Observabilidade

Stack proposta:

- Prometheus para métricas.
- Grafana para dashboards.

- OpenTelemetry para rastreamento distribuído.

Escalabilidade

O Horizontal Pod Autoscaler foi configurado para aumentar ou reduzir réplicas conforme consumo de CPU e memória, garantindo melhor utilização dos recursos e capacidade de resposta em períodos de alta demanda.

7. INFRAESTRUTURA COMO CÓDIGO

Foi criado um esqueleto Terraform para provisionamento da infraestrutura em nuvem.

Arquivos principais:

- providers.tf
- variables.tf
- main.tf
- outputs.tf

A utilização do Terraform garante reprodutibilidade, controle de mudanças e redução de configurações manuais.

8. CONCLUSÃO

A implementação do MVP permitiu aplicar conceitos modernos de Engenharia Cloud DevOps, utilizando Docker, Kubernetes, GitHub Actions e Terraform para construir uma plataforma preparada para escalabilidade, automação e alta disponibilidade.

FIM DO RELATÓRIO PRÁTICO